



AWS Vector Storage Choices: S3 Vectors, Amazon Aurora, and DynamoDB

Understanding Different Ways to Store
Embeddings, Operational Data, and AI
Context on AWS

Presented by: Akash Patel
Professor: Victor Govindaswamy
Concordia University Chicago



Why Do We Need Different Vector Storage Options?

- In previous Video, I learned how **Amazon OpenSearch can store embeddings and perform semantic or hybrid search**. But OpenSearch is not the only place where an AWS Generative AI application can keep vector data.
 - Different applications already store their information in different systems. One company may keep documents in Amazon S3, another may depend heavily on a relational database, and another may store live application data and chat history in DynamoDB.
 - Because of this, **the best place for vectors often depends on where the original data already lives and how the application needs to use it**. A vector store is not only about finding similar embeddings; cost, latency, filtering, transactions, operational data, and integration also matter.
 - In this video, I will study three different approaches: **Amazon S3 Vectors for large and cost-focused vector storage, Amazon Aurora with pgvector for relational data plus similarity search, and DynamoDB Vector Indexes for operational data and AI memory**.
-
- **Main idea:**
 - There is not one AWS storage design that fits every Generative AI application.

Amazon S3 Vectors: The Basic Idea

- **Amazon S3 Vectors allows vector embeddings to be stored in a purpose-built vector structure connected with Amazon S3.** Instead of creating and managing a full search cluster, the application can create a vector bucket and place vector indexes inside it.
- The course presents S3 Vectors as a **simpler and lower-cost approach for storing large collections of embeddings**, especially when every vector does not need extremely fast, high-frequency searching.
- To create the index, we define important properties such as the **number of embedding dimensions and the distance metric**. Those settings need to match how the embedding model represents and compares vectors.
- After the index exists, applications can store embeddings together with useful metadata and later send vector queries to find similar items.
- **Basic Structure**
- **S3 Vector Bucket**
 - ↓
 - Vector Index**
 - ↓
 - Vectors + Metadata**
- So S3 Vectors gives us vector-storage capability without requiring us to operate the same type of search infrastructure we discussed with OpenSearch.

What Is Stored Inside an S3 Vector Index?

- A vector record contains more than only a long list of numbers.
- **The key identifies the item represented by the vector.** For example, the key might represent a movie, product, document section, image, or another object.
- **The vector data contains the embedding itself.** In the course example, the embeddings are stored as floating-point values. Those numbers represent the meaning or characteristics learned by the embedding model.
- **Metadata provides extra information about the item.** For a movie embedding, metadata might include the original text and the genre. For a company document, metadata could instead contain the department, document type, year, or topic.
- **Simple Example**
- Imagine we store an embedding for:
- **Key:** Employee-Remote-Work-Policy
- **Vector:** Numerical embedding created by a Bedrock embedding model
- **Metadata:** Department = HR, Year = 2026, Topic = Remote Work
- This means the application can store semantic information and descriptive information together instead of keeping only the embedding.

Storing and Querying Vectors

- The course introduces two important operations.
- **put_vectors:**
- **put_vectors is used to add vector records into the index.** Each record can contain the vector, a key, and metadata that describes what that vector represents.
- Several vectors can be inserted together instead of making a separate request for every item. This becomes more efficient when an application is loading a large set of embeddings.
- **query_vectors:**
- **query_vectors searches the index using another vector.** Usually the application first converts the user's text, image, or other input into an embedding and then compares that query embedding with the stored vectors.
- **Simple Flow**
- **User Question**
 - Create Embedding
 - query_vectors
 - Find Similar Stored Vectors
 - Return Relevant Information
- The vector database does not answer the user's question itself. Its job is to **find information with similar meaning** so another system, such as a Foundation Model, can use that information.



S3 Vectors and Amazon Bedrock Knowledge Bases

- **Developers can create embeddings themselves and store them in S3 Vectors**, but the course also explains that S3 Vectors integrates with Amazon Bedrock Knowledge Bases.
 - With Knowledge Base integration, some of the vector-management process can be handled as part of the Bedrock workflow instead of the developer manually controlling every vector operation.
 - This connects with what I studied earlier about RAG: source information is prepared, embeddings are created, those vectors are stored, and relevant information is retrieved when the user asks a question.
 - The important difference in this video is the storage choice. Earlier I focused on **OpenSearch as the vector store**. Here the same general RAG requirement can use **S3 Vectors as the place where the embeddings are stored**.
 - **Example:**
 - A company may keep a very large archive of product manuals in S3. The manuals can be embedded and stored using S3 Vectors, and a Bedrock Knowledge Base can retrieve useful information when an employee asks a product-support question.
-

Performance, Consistency, and Tiered Search

- S3 Vectors is designed differently from OpenSearch, so the expected usage pattern is also different.
- **The course states that S3 Vectors is strongly consistent**, meaning newly added vector data becomes available for retrieval without waiting for a delayed consistency process.
- The vector data is also optimized over time for price and performance, but the course gives expected search performance in approximately the **100 millisecond to 1 second range** rather than positioning it as the fastest possible option.
- For applications where some searches are highly performance-sensitive, the course describes a **tiered search strategy**. Frequently searched vectors can use OpenSearch, while less frequently queried vectors can remain in S3 Vectors.
- S3 Vectors and OpenSearch can therefore be used together. However, the course warns that exporting data from S3 Vectors into OpenSearch can mean paying for storage in both places, so architecture and cost still need to be considered.
- **Easy Way to Remember**
- **S3 Vectors:** large-scale, simpler, cost-focused storage
OpenSearch: richer and more performance-focused search features
- This is not about one service always being better. They are designed for different requirements.

Moving from Vector Storage to Relational Data



- Not every AI application works mainly with documents.
- Imagine an AI assistant for an online store.
- The application may need to know:
 - Customer ID
 - Order number
 - Product price
 - Payment status
 - Shipping date
 - Inventory quantity
- This is **structured business data**, and companies often already store this kind of information in relational databases.
- That creates another question:
- **What if we need normal SQL data and vector similarity in the same application?**
- This is where services such as **Amazon RDS and Amazon Aurora** become important.

What Is Amazon RDS?



Amazon RDS stands for Relational Database Service. It is an AWS-managed database service for relational databases that use SQL.



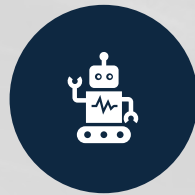
RDS supports several common database engines, including **PostgreSQL, MySQL, MariaDB, Oracle, Microsoft SQL Server, IBM Db2, and Amazon Aurora.**



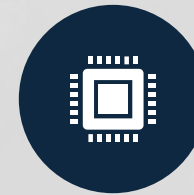
A relational database organizes structured information into tables with defined relationships. This makes it useful when an application needs predictable fields, SQL queries, joins, transactions, and structured business records.



In a Generative AI application, the Foundation Model may handle natural-language interaction, while RDS continues to store the reliable business information behind the application.



Example:
A customer can ask an AI assistant, **“What is the status of my latest order?”**



The AI may understand the question, but the actual order number, shipping state, and delivery information should come from the company's database rather than being invented by the model.

Why Use a Managed Database?

- Running a database manually means someone must also manage the infrastructure around that database.
- With RDS, AWS handles many operational tasks.
- The course covers managed capabilities such as **database provisioning, operating-system patching, backups, monitoring, maintenance, read replicas, and Multi-AZ disaster-recovery configurations.**
- This allows developers to focus more on the database and application instead of manually managing every part of the database server.
- **RDS Storage Auto Scaling can automatically increase database storage when free space becomes too low.** The user defines a maximum storage threshold so the database cannot grow without a defined limit.
- The course explains that automatic storage modification can occur when free storage remains below 10% for a period of time and other conditions are met. This is especially useful for applications where data growth is difficult to predict.
- **Main idea:**
- Managed does not mean the database needs no design. It means AWS takes responsibility for many infrastructure and maintenance tasks that would otherwise be handled manually.

Amazon Aurora as a Vector Store

- Amazon Aurora becomes especially interesting for Generative AI when using **Aurora PostgreSQL with the pgvector extension**.
- **pgvector adds a vector data type to PostgreSQL**. This allows embeddings to be stored in database columns alongside normal relational information.
- The database can calculate vector similarity using methods such as **cosine distance, L2 distance, and inner product**.
- It can also perform similarity searches using vector indexing methods while still supporting normal SQL operations.
- The main advantage is that **structured data and vector data can stay in the same database**.
- **Example**
- A product table could contain:
 - **Product_ID | Name | Price | Category | Description_Vector**
 - Now one system can use SQL to filter by category or price and also use the vector column to search for semantically similar product descriptions.
 - The course describes Aurora as potentially appropriate for **small or medium RAG systems where much of the information is already structured**.

Why Combine SQL Filtering with Vector Search?

- A vector search may find items with similar meaning, but sometimes similarity alone is not enough.
- Imagine a customer asks:
- **“Show me a lightweight laptop for programming under \$1,000.”**
- The application may need two different types of search.
- **Semantic Requirement**
- **“Lightweight laptop for programming”**
- An embedding can help find products whose descriptions match this meaning.
- **Structured Requirement**
- **Price < \$1,000**
- SQL can apply an exact numerical condition.
- This is why Aurora with pgvector can be useful.
- Vector similarity handles the **meaning-based part** of the request.
- SQL handles the **exact structured rules**.
- The result can combine both instead of keeping relational data in one place and embeddings in an entirely separate system.
- This type of design is especially useful when the application already depends heavily on PostgreSQL and needs vector capability without creating a completely separate data architecture.

What Is Amazon DynamoDB?

- Now we move from relational databases to a different type of database.
 - **Amazon DynamoDB is a fully managed NoSQL database.** Unlike RDS, it is not based on the traditional relational table-and-join model.
 - The course describes DynamoDB as a highly available distributed database that replicates data across multiple Availability Zones and can scale to very large workloads.
 - It is designed for **fast and consistent low-latency access** and integrates with AWS IAM for security and access management.
 - DynamoDB can also support event-driven architectures through **DynamoDB Streams**, which can react when information inside a table changes.
 - Because DynamoDB is designed for operational application data, it is commonly useful when the application needs information to be read and updated very quickly at scale.
-
- **Easy Difference**
 - **RDS / Aurora:** relational + SQL
 - **DynamoDB:** NoSQL + high-scale key-based operational access

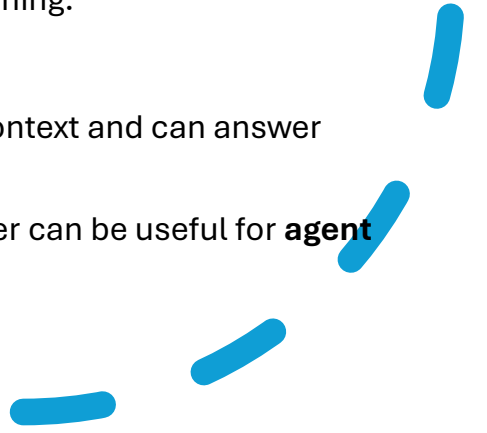
DynamoDB in Generative AI Applications

- The course connects DynamoDB directly with several GenAI use cases.
- **DynamoDB can provide near-real-time application information to an AI system.** For example, a chatbot may need access to the user's current account settings or recent application activity.
- It can also be used to **store and retrieve chat histories**. A chatbot can save previous messages so the application has a record of earlier conversations.
- The course describes this type of stored information as possible **long-term memory for AI agents**. Instead of remembering only the current message, an application can retrieve useful information from earlier interactions.
- This becomes important for **context awareness**. For example, if a customer previously told an assistant what product they own, the application may retrieve that stored information later instead of asking the same question again.
- **Example**
- **User ID → Previous Conversations → Saved Preferences → Current Request**
- DynamoDB can keep this operational context available so the AI experience does not always start from zero.

DynamoDB Vector Indexes

- DynamoDB can now go beyond storing normal application records.
- **Vector Indexes allow embeddings to be stored alongside the operational data already inside the DynamoDB table.**
- This means an application may not need to copy the same information into a completely separate vector database just to perform semantic similarity search.
- DynamoDB uses the **SearchVectors API** to perform Approximate Nearest Neighbor, or ANN, searches over the stored embeddings.
- The course lists three supported distance functions:
- **COSINE** – commonly appropriate for text embeddings
- **EUCLIDEAN** – compares straight-line distance between vectors
- **DOT_PRODUCT** – compares vectors using their mathematical dot product
- A vector index can also use a partition key and inline filters to narrow the search scope.
- **Projections control which table attributes are returned with the vector-search result**, similar in concept to selecting specific information that the application actually needs.
- Use cases mentioned in the course include **RAG, semantic search, AI-agent memory, and recommendation systems.**

Example: DynamoDB Vector Search for AI Memory

- Imagine an AI study assistant that talks with the same student many times.
 - The DynamoDB table could store:
 - **Student ID**
 - **Conversation text**
 - **Topic**
 - **Timestamp**
 - **Embedding**
 - Now suppose the student asks:
 - **“What was that AWS database topic I was confused about last week?”**
 - **What Happens?**
 1. The new question is converted into an embedding.
 2. The application calls **SearchVectors**.
 3. DynamoDB finds previous conversation items with similar meaning.
 4. The relevant messages are returned to the AI application.
 5. The Foundation Model receives those previous messages as context and can answer based on the student's earlier conversation.
 - This shows why keeping the vector and the application data together can be useful for **agent memory and context-aware assistants**.
- 

DynamoDB Vector Indexes: Important Limitations

- Vector support does not mean DynamoDB suddenly replaces every vector database.
- **The course states that vector indexes require on-demand capacity mode and use approximate ANN results.**
- Vector search is accessed through **SearchVectors** rather than normal **Query, Scan, PartiQL, or DAX** operations.
- Inline filters are more limited than the complex search and filtering capabilities available in a dedicated search platform.
- The response has size restrictions, and important choices such as the distance function and projection are fixed when the vector index is created.
- **Why Use It Anyway?**
- The major advantage is simplicity when **the operational data or chat history is already stored in DynamoDB.**
- Instead of maintaining:
- **DynamoDB → Synchronization Pipeline → Separate Vector Database**
- the application may be able to keep:
- **Operational Data + Embeddings → DynamoDB**
- This can reduce the need to maintain another copy of the same application information.



How the Storage Choices Fit Different Applications

- These services overlap in some capabilities, but their designs are different.
- **Amazon S3 Vectors**
 - Useful when the application needs to store **very large vector collections** and cost is an important concern, especially when the vectors are not searched constantly.
- **Amazon Aurora + pgvector**
 - Useful when the application already works with **relational PostgreSQL data** and needs SQL filtering, structured business information, and vector similarity in the same database.
- **Amazon DynamoDB Vector Indexes**
 - Useful when **operational data, chat histories, or AI-agent memory already live in DynamoDB** and the application also needs semantic recall without copying the information into another vector store.
- **Amazon OpenSearch**
 - From previous Video, OpenSearch remains useful when the application needs a richer **search platform**, including vector search, keyword search, hybrid retrieval, and more search-oriented control.
- **The important lesson is to start with the application's data and access pattern instead of choosing a service only because it supports vectors.** The course itself contrasts DynamoDB with S3 Vectors and OpenSearch based on where the data lives, latency, cost, and richness of search.



Key Takeaways & Thank You

- **What I Learned**
- **S3 Vectors provides a simpler way to store large collections of embeddings** using vector buckets and vector indexes, with metadata and integration with Bedrock Knowledge Bases.
- **Amazon RDS provides managed relational databases**, while Aurora PostgreSQL with pgvector allows normal relational records and vector embeddings to exist in the same database.
- **DynamoDB is a scalable NoSQL database that can support near-real-time application data, chat history, and AI-agent memory.**
- **DynamoDB Vector Indexes bring semantic similarity search closer to operational data**, which can reduce the need for a separate vector synchronization pipeline in some architectures.
- OpenSearch, S3 Vectors, Aurora, and DynamoDB can all participate in Generative AI systems, but **they solve different storage and retrieval problems.**
- The biggest lesson from this video is that **vector storage should fit the application's existing data architecture instead of being treated as an isolated AI feature.**
- **Thank You**
- **Prepared by: Akash Patel**
- **Concordia University Chicago**
- **Special thanks to Professor Victor Govindaswamy for his guidance and support throughout this course. Studying these services together is helping me understand that Generative AI architecture is not only about choosing a Foundation Model. The way we store normal business data, embeddings, chat history, and application context also affects how reliable and practical the final AI system can be. I truly appreciate his guidance in helping us connect the AWS concepts with complete real-world application designs.**